LABORATORY REPORT

CIP-B101: Basic Computer Skills for Digital Forensics

Lab 1A: Number Systems, ASCII, Timestamps and Data Representation

Institution: International Cybersecurity and Digital Forensics Academy (ICDFA)

Student Name: Marie Dogbey

Student ID: C11/26/PCDFIR/17028

Platform used: Kali GNU/Linux 2026.2(python 3.12, xxd, bc)

2. Task A - Number Systems and Manual Conversion

2.2 Manual Conversion Worksheets

1. Convert 101010_2 to decimal manually. Show the weight of each bit from right to left.

   $0 \times 2^0 = 0$
   $1 \times 2^1 = 2$
   $0 \times 2^2 = 0$
   $1 \times 2^3 = 8$
   $0 \times 2^4 = 0$
   $1 \times 2^5 = 32$
   $0+2+0+8+0+32 = 42_{10}$

2. Convert $AB2_{16}$ to decimal manually. Remember that A = 10 and B = 11.

   $2 \times 16^0 = 2x1 = 2$

   $B \times 16^1 = 11x\ 16 = 176$

   $A \times 16^2 = 10x256 = 2560$

   $2+176+2560 = 2738$

3. Convert 126_{10} to binary manually by division by 2.

   $126 \div 2 = 63$ remainder 0

   $63 \div 2 = 31$ remainder 1

   $31 \div 2 = 15$ remainder 1

   $15 \div 2 = 7$ remainder 1

   $7 \div 2 = 3$ remainder 1

   $3 \div 2 = 1$ remainder 1

   $1 \div 2 = 0$ remainder 1

   $= 1111110_2$

4. Convert 2738_{10} to hexadecimal manually by division by 16.

   $2738 \div 16 = 171$ remainder 2(2)

   $171 \div 16 = 10$ remainder 11(Hex B)

   $10 \div 16 = 0$ remainder 10 (Hex A)

= $AB2_{18}$

5. Convert 101010110010_2 to hexadecimal by grouping bits in fours from the right.

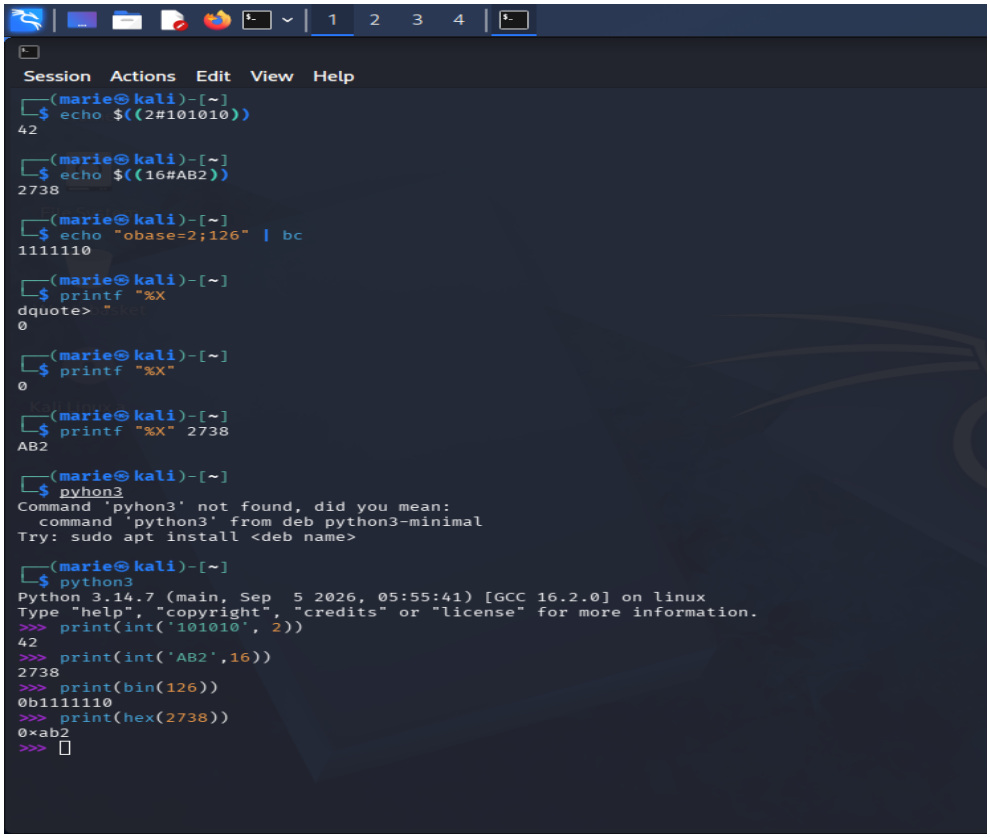
$1010_2 = 0x2^2 + 1x2^1 + 0x2^0 + 1x\ 2 = 8+0+2+0 = 10$=Hex A

$1011_2 = 1x2^3 + 1x2^2 + 0x2^1 + 1x2^0 = 8+0+2+1=11$ =Hex B

$0010_2 = 0x2^3 + 1x2^2 + 0x2^1 + 0x2^0 = 0+0+2+0 = 2$ =2 =$AB2_{16}$

TASK 2.3

Validate With Linux Shell and Python



Evidence Value	Given Base	Manual Answer	Command Used to Verify	Screenshot Ref.
101010	Binary	42	echo $((2#101010))	
AB2	Hexadecimal	2738	echo((16#AB2))	
126	Decimal	1111110	echo “obase=2;126” | bc	
2738	Decimal	AB2	“%X” 2738	
101010110010	Binary	$AB2_{16}$		

```
┌──(marie㉿kali)-[~]
└─$ echo $((2#101010))
42
```

SCREENSHOT 2.1

```
┌──(marie㉿kali)-[~]
└─$ echo $((16#AB2))
2738
```

SCREENSHOT 2.2

```
┌──(marie㉿kali)-[~]
└─$ echo "obase=2;126" | bc
1111110
```

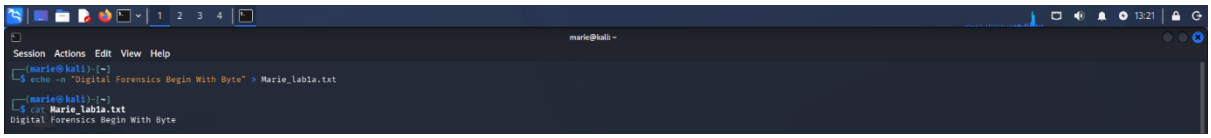
SCREENSHOT 2.3

```
┌──(marie㉿kali)-[~]
└─$ printf "%X" 2738
AB2
```

SCREENSHOT 2.4

3.4 Student Exercise

6. Create a file named yourfirstname_lab1a.txt containing the text: Digital Forensics Begins With Bytes.



7. Use xxd to display the file content in hexadecimal.

```
┌──(marie㉿kali)-[~]
└─$ xxd -p Marie_lab1a.txt
4469676974616c20466f72656e7369637320426567696e20576974682042
797465
```

8. Extract the plain hex using xxd -p.

```
┌──(marie㉿kali)-[~]
└─$ xxd -p Marie_lab1a.txt
4469676974616c20466f72656e7369637320426567696e20576974682042
797465
```

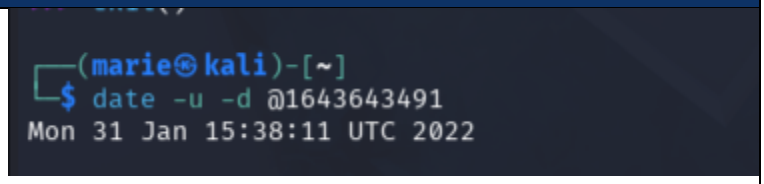
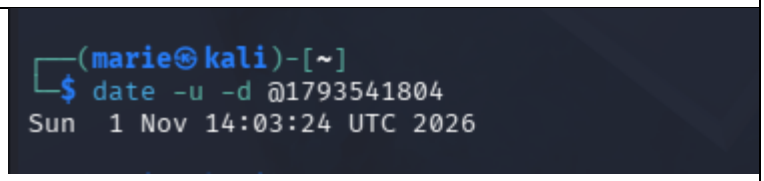
9. Convert the plain hex back to readable text.



10. Explain why 0A may appear at the end of output when echo is used without -n.
    In ASCII table, a newline has the decimal value of 10, which is 0A in hexadecimal.

The 0A byte represents the ASCII Line Feed character. Standard ‘echo’ appends this automatically for terminal formatting while ‘echo -n’ suppresses it to write the pure string elements only.

Timestamp Worksheet

Timestamp Value	Format	Conversion Command	Readable UTC Result	Screenshot Ref.
1643643491	Unix seconds	Date -u -d @1643643491	Mon 31 Jan 15:38:11 UTC 2022	
1675008832	Unix seconds	Date -u -d @1675008832	Sun 29 Jan 16:13:52 UTC 2023	
725846400	Mac Absolute Time seconds	Date -u -d @725846400	Fri 2 Jan 00:00:00 UTC 199	
Current time	Unix seconds	Date -u -d @1793541804	Sun 1 Nov 14:03:24 UTC 2026	

5 Task D – Hashing and the Newline Problem

11. Create two files containing your first name: one using echo and one using echo -n.



12. Use xxd to compare the byte-level difference.



13. Calculate MD5 and SHA-256 for both files.



14. Explain why the hash values differ.

The hash value differs because even the smallest change in the input data can result in a completely different output hash. Because the actual bytes processed by the hashing algorithm are different, this is why the results are different also.

15. State which commands you would use if you needed to hash exactly the visible characters only.

    Command to use: For MD5 echo -n “marie” | md5sum

    For SHA-256: echo -n “marie” | sha256sum

6. Task E - Endianness and Data Stored in Memory

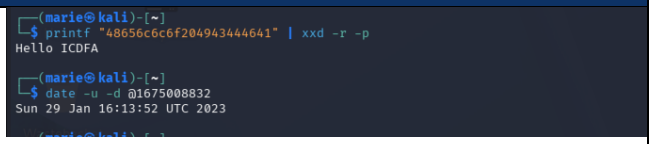
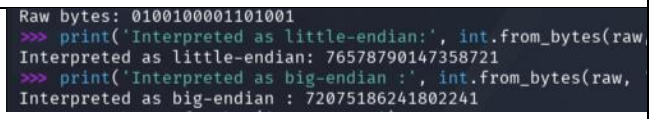
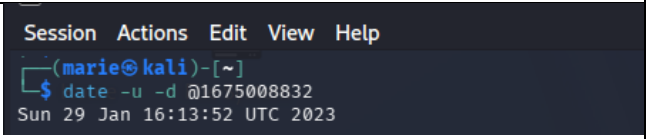
Endianess governs the directional storage pattern of numerical components across memory pages .

Raw Hex Bytes	Little-Endian Decimal	Big-Endian Decimal	Command Used	Explanation
01000000	1	16777216	python3 -c "b = bytes.fromhex('01000000'); print(int.from_bytes(b, 'little'), int.from_bytes(b, 'big'))"	LE places LSB first; BE places MSB first.
FF000000	255	4278190080	python3 -c "b = bytes.fromhex('FF000000'); print(int.from_bytes(b, 'little'), int.from_bytes(b, 'big'))"	LE interprets FF as 255; BE expands it outward.
00000100	65536	256	python3 -c "b = bytes.fromhex('00000100'); print(int.from_bytes(b, 'little'), int.from_bytes(b, 'big'))" int.from_bytes()	LE marks 01 as 2^16; BE treats 01 as 2^8.
78563412	305419896	2018915346	python3 -c "b = bytes.fromhex('78563412'); print(int.from_bytes(b, 'little'), int.from_bytes(b, 'big'))"	Classic ordering flip sequence example.

```
>>> print('Interpreted as big-endian :', int.from_bytes(raw, 'big'))
Interpreted as big-endian : 16777216
>>> raw = bytes.fromhex('FF000000')
>>> print('Raw bytes:', raw.hex())
Raw bytes: ff000000
>>> print(Interpreted as little-endian:', int.from_bytes(raw, 'little'))
  File "<python-input-13>", line 1
    print(Interpreted as little-endian:', int.from_bytes(raw, 'little'))

SyntaxError: unterminated string literal (detected at line 1)
>>> print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))
Interpreted as little-endian: 255
>>> print('Interpreted as big-endian :', int.from_bytes(raw, 'big'))
Interpreted as big-endian : 4278190080
>>> raw = bytes.fromhex('00000100')
>>> print('Raw bytes:', raw.hex())
Raw bytes: 00000100
>>> print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))
Interpreted as little-endian: 65536
>>> print('Interpreted as big-endian :', int.from_bytes(raw, 'big'))
Interpreted as big-endian : 256
>>> raw = bytes.fromhex('78563412')
>>> print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))
Interpreted as little-endian: 305419896
>>> print('Interpreted as big-endian :', int.from_bytes(raw, 'big'))
Interpreted as big-endian : 2018915346
>>> raw = bytes.fromhex('48656c6c6f204943444641')
>>> print('Raw bytes:', raw.hex())
Raw bytes: 48656c6c6f204943444641
>>> print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))
Interpreted as little-endian: 7891200315579092054392148O
>>> print('Interpreted as big-endian :', int.from_bytes(raw, 'big'))
Interpreted as big-endian : 875216180888824729213517 45
>>> raw = bytes.fromhex('0100100001101001')
>>> print('Raw bytes:', raw.hex())
Raw bytes: 0100100001101001
>>> print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))
Interpreted as little-endian: 76578790147358721
>>> print('Interpreted as big-endian :', int.from_bytes(raw, 'big'))
Interpreted as big-endian : 72075186241802241
>>> raw = bytes.fromhex('1675008832')
>>> print('Raw bytes:', raw.hex())
Raw bytes: 1675008832
>>> print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))
Interpreted as little-endian: 217030096150
>>> print('Interpreted as big-endian :', int.from_bytes(raw, 'big'))
Interpreted as big-endian : 96452249650
```

7. Task F - Mini Forensic Interpretation Case.

Evidence Item	Value	Student Interpretation	Command/Screenshot Ref.
EVID-001	48656c6c6f204943444641	Value was decoded into 'Hello ICDFA'.	
EVID-002	0100100001101001	Binary conversion to text characters: 'Hi' . which is essential for low-level stream analysis or protocol layer mining.	
EVID-003	1675008832	Sun 29 Jan 16:13:52 UTC 2023. Epoch time serves as a metrics for building timelines.	
EVID-004	725846400 (Mac Absolute Time)	FRI 1 Jan 00:00:00 UTC 1993	

Evidence Item	Value	Student Interpretation	Command/Screenshot Ref.
EVID-005	78563412 as little-endian and big-endian	Little Endian = 305419896. Big Endian = 2018915346	

7.1 CASE NARATIVE

Evidence 001 and 002 represent the text data layer these elements provide context in terms of communication strings. EVID-003 and EVID-004 establishes structured event boundaries. translating these parameters allows cross-log event synchronization. Evidence 005 demonstrates physical representation constraints. It is important to identify system byte-ordering properties before mapping hex dumps to decimal sizes too preserve structural evidence validity.

Student Reflection:

Number systems and timestamps form the foundation of digital forensics, It enables investigators to translate raw machine data into verifiable events. Digital evidence exists as raw binary, octal and hexadecimal machine code. An understanding of the number systems is essential for decoding file headers, navigating memory structures and manually extracting hidden data.

8.2 Minimum Screenshot Checklist

No.	Screenshot Evidence Description	Included in Report?
1	Working folder creation and pwd output verification	Yes [Verified]
2	Binary/hex/decimal manual and pipeline verification commands	Yes [Verified]
3	xxd hex array output for generated text assets	Yes [Verified]
4	Hex-to-ASCII clean text translation validation pipelines	Yes [Verified]
5	Unix epoch timestamp string translation output data	Yes [Verified]
6	Mac Absolute Time system offset translation validations	Yes [Verified]

7	Hash mismatch confirmation using md5sum and sha256sum profiles	Yes [Verified]
8	Endian orientation calculation via Python script systems	Yes [Verified]
9	Mini-case item processing and structural command workflows	Yes [Verified]